1. you are building a food delivery application. At which stage would you prefer using a compiler over an interpreter, and why? Explain based on execution speed, error detection and deployment
   ans. For a food delivery application, I would prefer using a **compiler during the production/deployment stage** because compiled programs provide better performance, error checking, and easier distribution.

Factor	Compiler	Interpreter
Execution Speed	Faster because code is translated into machine code before execution.	Slower because code is translated line by line during execution.
Error Detection	Detects many errors during compilation before the program runs.	Errors are found only when the problematic line is executed.
Deployment	Compiled executable can be distributed without source code.	Requires the interpreter to be installed on the target machine.

   Food Delivery App Example
   Development Stage
   During development, an **interpreter** can be useful because:

- Code can be tested quickly.
- Changes can be seen immediately.
- Debugging is easier.

   Example: Testing restaurant search, cart functionality, or order tracking.
   Production Stage
   For the final deployed application, a **compiler** is preferred because:

- Faster response times for users placing orders.
- Better performance when handling thousands of customers simultaneously.
- Many errors are caught before release.
- Easier and more secure deployment.

2. You are asked to design a program that calculates employee salary with multiple conditions (bonus, tax, overtime). Which programming constructs would you combine to solve this efficiently?
   Ans.
   I am creating this program to like the use in conditions
   1.variables

2.input output statement use for printing the data
3. condition statement use like if else else-if .
4. arithmetic operator use like +,-,%.
5. and last functions and loops use . and create this program
Gross Salary = Basic Salary + Bonus + Overtime Pay
Net Salary = Gross Salary – Tax

3. While developing a menu-driven console application, why are loops preferred over conditional repetition? Explain which loop you would choose and why?
Ans.
 Reduces code repetition – The menu is written once and executed multiple times.
 Improves readability – The program becomes easier to understand and maintain.
 Allows continuous user interaction – Users can perform multiple operations without restarting the program.
 Makes the program scalable – New menu options can be added easily.

Which Loop Would I Choose.
For a menu-driven program, I would choose a **do-while loop**.
Reason:

- The menu should be displayed **at least once**.
- The condition is checked **after** the menu is executed.
- It is ideal for programs that continue until the user selects "Exit".

```
#include<iostream>

using namespace std

int main()

{

   int choice;

   do

   {

      cout << "\n----- MENU -----" << endl;

      cout << "1. Add" << endl;

      cout << "2. Delete" << endl;

      cout << "3. Exit" << endl;

      cout << "Enter choice: ";

      cin >> choice;
```

```
        switch(choice)
        {
            case 1:
                cout << "Add operation" << endl;
                break;
            case 2:
                cout << "Delete operation" << endl;
                break;
            case 3:
                cout << "Exiting..." << endl;
                break;
            default:
                cout << "Invalid choice" << endl;
        }
    } while(choice != 3);
    return 0;
}
```

4. A student writes a program to calculate average marks using int variables and gets incorrect output. Identify the probable cause and explain how choosing the correct data type solves the issue.
   Ans.
   The student is using int variables to calculate the average. In C++, when two integers are divided, the result is integer division, which discards the decimal part.

   The int part in only store in number no add a decimal (0.55) number.
   So you can use float and double variable.

5. You are debugging a program that complies successfully but crashes at runtime. Which category of error is this? Mention two debugging practices you would follow to resolve it.

Ans.

Category of Error

If a program **compiles successfully** but **crashes while running**, it is a **Runtime Error**. Runtime errors occur after compilation during program execution. Common causes include:

- Division by zero
- Accessing invalid array indexes
- Null/dangling pointers
- Memory allocation issues
- Infinite recursion

1. Use Debugging Statements (Print Statements)

Add cout statements at different points in the program to identify where the crash occurs.

2. Use a Debugger and Check Variable Values

Run the program in a debugger (such as the debugger in Visual Studio or Code::Blocks) and:

- Set breakpoints
- Execute the program step by step
- Inspect variable values
- Check memory and pointer references

This helps identify issues like invalid memory access or unexpected values.

6. You need to store student records with name, marks, and grade. Why would you choose a structure over multiple arrays in this scenario?

   Ans.

   When storing student records containing **name, marks, and grade**, a **structure (struct)** is a better choice because it groups related data into a single unit.

   Advantages:

- All information about a student is stored together.
- Easier to read and manage.
- Reduces chances of errors.
- Makes the program more organized and scalable.

```
#include<iostream>
using namespace std;

struct Student
{
   string name;
   int marks;
```

```
    char grade;
};

int main()
{
    Student s;

    s.name = "Rahul";
    s.marks = 85;
    s.grade = 'A';

    cout << "Name: " << s.name << endl;
    cout << "Marks: " << s.marks << endl;
    cout << "Grade: " << s.grade << endl;

    return 0;
}
```